

基于 ESP32 的 AI 辅助数字钢琴与 AI 原生开发工具链

—— 最终技术报告 ——

项目名称	基于 ESP32 的 AI 辅助数字钢琴与 AI 原生开发工具链
------	----------------------------------

课程 25 级实验班·单片机综合实验（暑假大作业）

小组成员 王子睿、徐天翔、蒲正杰、崔江涛、王因畅（第四小组）

项目周期 2026 年 7 月 6 日 — 8 月 9 日（共 5 周）

相关仓库 github.com/ChuOZhen/MCPiano
（本报告载体，数字钢琴 + 工具链）
github.com/WANG5294/Yuanshen
（小组智能体协作项目）

格式说明：本报告正文约 15—20 页（A4，宋体小四，1.5 倍行距），所有图、表均有编号并在正文中引用。报告中涉及的具体文件路径均可在 GitHub 仓库中对应查找。

1 引言

1.1 项目背景与目标

嵌入式系统开发长期以人工编写代码、手动烧录、反复调试为主要工作模式。近年来，以 Claude Code、Codex、Zcode 等为代表的大语言模型（LLM）与 AI 编程助手极大降低了编程门槛，但存在一个显著短板：**AI 无法直接与物理硬件交互**。AI 生成一段 ESP32 的 MicroPython 程序后，工程师仍需手动上传、重启、观察串口、收集日志、诊断故障，再往复迭代——AI 被限制在纯软件层面，嵌入式开发中最重要的硬件闭环无法自动化。

本项目引入一种全新的工程范式——**AI 原生嵌入式开发**：不仅让 AI 辅助生成代码，更让 AI 能够直接与嵌入式硬件通信，将「代码生成 → 自动部署 → 执行观测 → 日志反馈

→ AI 诊断与修复 → 再次迭代」整合为一条连续的工程管线。项目包含两个相互支撑的任务：

- **任务一（数字钢琴）**：利用 ESP32 开发板实现一台功能完整的 9 键数字钢琴（按键输入 → 音符发声 → LED 反馈；共 10 个面包板按键：7 音阶 + 2 八度 + 1 录制/播放），并扩展录制/回放、八度切换等进阶功能。
- **任务二（AI 原生开发工具链）**：设计并实现一套基于 MCP (Model Context Protocol) 的工具链，使 AI 编程助手能直接与 ESP32 交互（文件传输、程序执行、串口监控、错误报告等），并以数字钢琴为验证基准，演示 AI 通过工具链驱动嵌入式开发的完整闭环。

1.2 两个任务的关系

两个任务相互支撑、不可分割。数字钢琴是**应用载体**，工具链是**工程基础设施**：工具链使 AI 能够“触碰”硬件，数字钢琴则是检验工具链有效性的工程基准。本项目最终验证目标是：亲眼看到 AI 通过工具链自主完成一次「发现问题 → 修改代码 → 重新部署 → 验证通过」的迭代。

此外，小组在第四周进一步将工具链思想延伸到智能体层面，自主设计了闭环自动化智能体 **Yuanshen**（约 4479 行 Python），实现自然语言 → 需求规范化 → 接线确认 → 代码编写 → 固件烧录 → 实机测试 → 结果汇报的完整自动化流程，并以 npm 包 `yuanshen-esp32-agent` 发布。详见第 6 章。

1.3 报告结构

本报告按任务书建议结构组织：第 2 章硬件平台分析；第 3 章任务一数字钢琴设计与实现；第 4 章任务二工具链设计与实现；第 5 章闭环验证与分析；第 6 章 AI 协作反思；第 7 章总结与展望；附录给出周度开发历程、测试清单与参考资料。

2 硬件平台分析

2.1 硬件架构与功能模块

开发板核心为 **ESP32-D0WD-V3** 双核微控制器（QFN-48 封装），板载 CP2102N USB 转串口芯片、32 Mbit SPI Flash（ZB25VQ32）、40 MHz 晶振。电源链为 USB 5V → AMS1085-5V → AMS1085-3.3V 两级稳压，前端带 500 mA 自恢复保险丝与防反接二极管（详见 `hardware/bom_analysis.md` 的 38 项 BOM 分析）。

板载/外接功能模块包括：10 个面包板按键、7 个音调键 LED + 2 个八度键 LED（硬件直驱）、板载红绿 LED（GPIO32/33）、MAX98357A I2S 功放 + 小喇叭、以及原始设计中曾接入、后因硬件故障移除的 SSD1306 OLED 显示屏。

2.2 GPIO 分配与接口分析

经对原理图、PCB 版图与 BOM 的交叉分析（`hardware/bom_analysis.md`），最终 GPIO 分配如表 2-1 所示。

表 2-1 数字钢琴 GPIO 分配表

功能	GPIO	类型	说明
KEY do / re / mi / fa / sol / la / si	23 / 22 / 21 / 19 / 18 / 14 / 12	输入	7 音阶键，按下 = 低电平
KEY 八度+ / 八度-	34 / 35	输入	INPUT-ONLY 引脚，需外接上拉
KEY 录制/播放	5	输入	内部上拉可用，短按录制、长按播放
LED（7 音键 + 2 八度键）	—	直驱	3.3V → LED → 330Ω → 按键 → GND
板载 LED（绿 / 红）	32 / 33	输出	低电平点亮，指示八度与录制/播放状态
I2S BCLK / LRCK / DIN	16 / 17 / 25	输出	连接 MAX98357A 功放
串口	USB（CP2102N）	—	115200 baud, /dev/ttyACM0

注意事项（均有实测依据）：

- GPIO 6–11 被 SPI Flash 占用，不可使用（`hardware/bom_analysis.md`）。
- GPIO 34/35 为 INPUT-ONLY 引脚，内部上拉无效，必须外接上拉电阻；本项目曾因此出现八度键浮空误触发，通过 `tests/diag_octave_pins.py` 100 次采样诊断脚本定位后，以外部 10kΩ 上拉解决。
- MAX98357A 接线存在两个易错点（README 中特别标注）：① GAIN 逻辑各模块不统一，本模块 GND=高增益、VIN=正常增益，误接 GND 会导致小音量削顶；② SD_MODE 必须接 VIN，悬空会导致功放状态不稳定，表现为大音量反而变小、间歇失声。

2.3 数字钢琴的硬件资源选用及理由

- 音频输出采用 MAX98357A I2S 功放 + 喇叭，替代初版 PWM 蜂鸣器。理由：方波含大量高次谐波、听感尖锐，I2S 支持 16-bit PCM 波形、音准干净，且为第四周引入真实钢琴 WAV 采样预留接口（详见 3.5 节）。
- 音调键 LED 采用硬件直驱（按键回路串联 LED），而非 GPIO 软件驱动，实现零延迟声光同步（详见 3.6 节）。

- GPIO 5 用于录制/播放键：内部上拉可用、支持长按区分两种语义，无需额外上拉元件。

2.4 未使用资源说明

数字钢琴仅使用按键、LED、声音输出（MAX98357A I2S 功放）三类外设，其余板载资源按如下情况处理（表 2-2）：

表 2-2 未使用的板载资源及原因

板载资源	处理	原因说明
SSD1306 OLED (I2C)	尝试后移除	第二周多轮调试（GPIO2/4→5/13 迁移、降频 100 kHz、外接上拉）确认板载 OLED 硬件故障（I2C 无响应），W2 未移除（commit 48d07f4）
蜂鸣器 MLT-5020 (PWM)	被取代	W2 起改为 MAX98357A I2S 功放播放正弦波/采样，音质与扩展性远超 PWM 方波；相关测试保留作参考
板载 KEY1/KEY2、红绿 LED	部分使用	KEY1/KEY2（GPIO35/34）作为八度切换键使用；红绿 LED（GPIO32/33）作为八度方向与录制/播放状态指示
ADC / 触摸 / SPI / SD / MPU6050	不使用	与数字钢琴需求无关；未使用资源后续在 Yuanshen 智能体的硬件能力 Skill 中覆盖

3 任务一：数字钢琴设计与实现

3.1 软件架构与模块划分

固件基于 MicroPython v1.23.0，采用分层模块化设计，共 4 个模块（表 3-1），主循环以 5 ms 周期轮询驱动。

表 3-1 数字钢琴固件模块划分

模块	行数	职责
piano/main.py	18	入口：实例化 Piano，while True: piano.tick(); sleep_ms(5)，捕获 KeyboardInterrupt 优雅退出
piano/piano.py	176	钢琴状态机：八度切换、录制/回放状态机、LED 指示、事件驱动发声
piano/buttons.py	194	按键扫描：软件去抖、边沿检测、实时状态查询、长按计时
piano/i2s_audio.py	82	I2S 音频驱动：总线初始化、WAV 流式回放、音符映射

图 3-1 给出系统数据流：按键电平 → ButtonController 去抖/边沿 → Piano.tick() 状态机 → I2SAudio.play_note() → I2S 总线 → 功放发声。



图 3-1 数字钢琴系统数据流

3.2 按键扫描与去抖

`buttons.py` 实现非阻塞软件去抖： `_update_states()` 记录各引脚原始电平变化时刻，仅当电平稳定满 20 ms 才更新去抖后的稳定状态，避免机械抖动导致的重复触发。

关键设计决策（`buttons.py:127-134`）：

- 音阶键走稳定去抖状态（`is_pressed()`），保证单次按压只触发一次发声；
- 八度键直接返回实时电平——按住生效的功能键不需要去抖，减少响应延迟；
- 录制/播放键利用按下沿启动长按计时（800 ms 阈值），区分短按（开始/停止录制）与长按（播放）。

去抖状态查询核心逻辑（节选自 `buttons.py` 的 `is_pressed`，已省略通用化处理）：

```

def is_pressed(self, name: str) -> bool:
    """查询指定按键当前是否处于稳定按下状态。"""
    now = time.ticks_ms()

    if name in FUNC_GPIO:
        # 八度键是按住生效的功能键，不需要去抖，直接返回实时电平
        return self._func_pins[name].value() == 0
    elif name in NOTE_NAMES:
        idx = NOTE_NAMES.index(name)
        pin = self._note_pins[idx]
    else:
        return False

    # 音阶键：电平变化时记录变化时刻，稳定满 debounce_ms(20 ms)
    # 才更新稳定状态，实现非阻塞去抖
    val = pin.value()
    if val != self._note_raw[idx]:
        self._note_raw[idx] = val
        self._note_changed_at[idx] = now
    if val != self._note_stable[idx]:
        if time.ticks_diff(now, self._note_changed_at[idx]) >= self.debounce_ms:
            self._note_stable[idx] = val
    return self._note_stable[idx] == 0

```

3.3 八度切换

piano.py:54-64 维护 `octave_shift ∈ {-1, 0, +1}`，采用边沿触发 **toggle** 逻辑：按一次八度+进入高八度 (+1)，再按一次归零；八度-同理。文件选择索引 `idx = octave_shift + 1` 映射到低 (C3) / 中 (C4) / 高 (C5) 三档采样文件。板载 LED 提供视觉反馈：绿灯 (GPIO32) = 高八度，红灯 (GPIO33) = 低八度，空闲时按当前八度方向点亮。

3.4 录制/回放状态机

piano.py:87-162 实现三态状态机 (IDLE / RECORDING / PLAYING)：

- **录制**：GPIO5 短按进入录制，边沿触发 `note_on` 时记录事件 `{type, note, octave, time}`，其中 `time` 为相对录制起点的 ticks 差值——只记录按下事件，不记录释放，避免时间戳漂移；
- **回放**：长按 (≥ 800 ms) 进入播放，按事件时间戳到点逐个调用 `audio.play_note()`，播完自动回到 IDLE；`_record_long_triggered` 标志保证长按只触发一次播放、释放时不误切录制；回放期间对录制列表做快照，防止并发修改。
- LED 语义：录制中绿灯常亮，播放中红灯常亮。

3.5 I2S 音频播放：真实钢琴采样回放

这是 W4 的核心升级 (commit ee2e148)。此前 W2 版本用软件生成正弦波 PCM (test_max98357a.py)，W4 改为**真实钢琴 WAV 采样流式回放**：

- **采样集**：samples/ 含 21 个 WAV (A3-G5, 7 音名 × 3 八度)，命名规律为「音名 (do→C) + 八度数字 (3/4/5)」，如 c3.wav 为低八度 do、A4.wav 为国际标准音 la。每个文件 80044 字节 = 44 B WAV 头 + 80000 B PCM，即 2.5 s @ 16 kHz / 16-bit / 单声道。
- **流式播放** (i2s_audio.py:44-82)：读取 44 B 头部并校验 RIFF 魔数 (非法直接返回) → 以 **512 B 分块** 循环写入 I2S (奇数长度块截为偶数，保证 16-bit 对齐) → 累计达到 MAX_PLAY_BYTES = 24000 (约 0.75 s @ 16 kHz) 即截断，避免采样尾音拖沓。
- **低内存占用**：一次仅保留 512 B 缓冲，配合 ibuf=4000 硬件 DMA 缓冲，从 Flash 流式读取，RAM 占用极低 (README:329)。
- **I2S 配置**：SCK=GPIO16、WS=GPIO17、SD=GPIO25，TX 模式、16-bit、单声道、16000 Hz。
- **音符映射表**：do→C, re→D, mi→E, fa→F, sol→G, la→A, si→B，与按键 GPIO 元组、音符名元组三层严格对齐。

流式播放核心循环 (节选自 i2s_audio.py 的 play_note)：

```
def play_note(self, note: str, octave: int = 0):
    """播放采样, octave: -1=低八度, 0=中音, 1=高八度"""
    path = self._get_path(note, octave)
    if not path:
        return
    try:
        stat = os.stat(path)
    except OSError:
        return
    self._stopped = False
    try:
        with open(path, 'rb') as f:
            header = f.read(44)          # 跳过标准 44 B WAV 头
            if len(header) < 44 or header[:4] != b'RIFF':
                return
            total = 0
            while total < MAX_PLAY_BYTES:  # 24000 B ≈ 0.75 s 截断
                if self._stopped:
                    break
                chunk = f.read(512)        # 512 B 分块流式写入
                if not chunk:
                    break
                if len(chunk) % 2 != 0:
                    chunk = chunk[:-1]    # 偶数对齐, 保证 16-bit
                self.i2s.write(chunk)
                total += len(chunk)
    except Exception as e:
        print(f"[音频] 错误: {e}")
```

每次仅保留 512 B 缓冲，Flash 流式读取，内存占用极低。

设计权衡：`play_note()` 为阻塞式写入，多键同时按下时按顺序覆盖播放（"播放最后按下音"）；阻塞期间主循环不响应其他按键，换取实现简单性与声音完整性，在演示场景中可接受。

3.6 LED 声光同步：硬件直驱设计

W3 的关键设计：7 个音调键 + 2 个八度键的 LED 从软件 GPIO 驱动改为**按键回路硬件直驱**——`3.3V → LED → 330Ω → 按键 → GND`。GPIO 只做输入检测、不驱动 LED，因此按下瞬间物理点亮、释放瞬间熄灭，**零软件延迟**，消除了此前软件输出带来的声光不同步。这一改动同时删除了 `piano/leds.py` 模块，简化了固件。

3.7 扩展功能汇总

数字钢琴的扩展功能汇总见表 3-2。

扩展功能	实现要点	引入周次
八度切换（三档）	GPIO34/35 边沿 toggle，LED 绿/红指示	W2 初版 / W4 采样版
录制/回放	GPIO5 短按录制、长按播放，事件时间戳回放	W3
真实钢琴音色	21 个 WAV 采样 512B 分块流式回放	W4
LED 硬件直驱声光同步	按键回路串联 LED	W3

表 3-2 数字钢琴扩展功能一览

3.8 系统测试

`tests/` 下 9 个测试脚本覆盖从外设到系统的各级验证（详见附录 B）：

- W1 外设级：`test_button.py`（GPIO34/35 电平与边沿）、`test_led.py`（GPIO32/33）、`test_buzzer.py`（PWM 蜂鸣器，后已被 I2S 功放取代）；
- W2 系统级：`test_buttons_9key.py`（9 键矩阵）、`test_max98357a.py`（I2S 功放）、`test_piano_v1.py`（钢琴 v1 集成，10s 随机按键稳定性）；
- W3/W4 功能级：`test_octave_with_led.py`（八度键与 LED 联动）。

如实记录：`test_piano_v1.py` 仍引用 W2 旧版 `Piano` API（`play_note(idx)`、`handle_events()` 等），与 W4 采样版固件接口不一致，直接运行会 `ImportError`；README 将其列为快速开始用例属文档未同步，已在第 6 章 AI 协作反思中讨论此问题。

4 任务二：AI 原生开发工具链设计与实现

4.1 工具链架构设计

工具链采用 **MCP (Model Context Protocol) 服务器模式**：AI 编程助手 (KimiCode) 作为 MCP 客户端，通过标准 JSON-RPC 2.0 协议与本项目开发的 MCP 服务器通信，服务器把 6 个硬件操作能力封装为工具供 AI 调用。整体架构如图 4-1 所示。

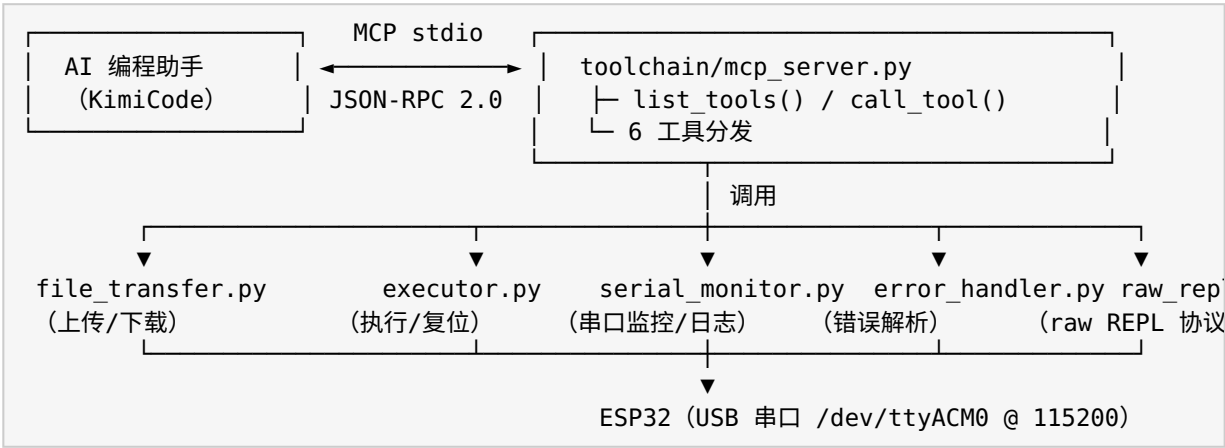


图 4-1 MCP 工具链架构

4.2 MCP 协议与服务器实现

`mcp_server.py` 基于官方 `mcp.server.Server` + `stdio_server`，进程名 `mcplaneo-esp32`。两个核心装饰器：

- `@app.list_tools()`：声明工具名、描述与 JSON Schema 参数定义；
- `@app.call_tool()`：按工具名分发到后端实现，结果统一 `json.dumps(..., ensure_ascii=False)` 打包为 `TextContent` 返回。

实际注册 6 个工具：`esp32_upload`、`esp32_execute`、`esp32_reset`、`esp32_serial`、`esp32_logs`、`esp32_error`。需要说明：`esp32_download` 的后端 (`file_transfer.py:76-119`) 已实现并导入，但未注册进工具列表，README 工具表列 7 项而实际为 6 项，KimiCode 加载实测显示 "6 tools connected"——此为如实记录的实现与文档差异，计划在后续版本注册该工具。

4.3 各工具模块详细实现

4.3.1 raw REPL 协议层 (`raw_repl.py`，142 行)

这是工具链与 MicroPython 设备通信的协议核心，实现标准 MicroPython raw REPL 交互：

- **握手序列**：`\x03` (Ctrl-C 中断运行程序) → 100 ms → `\x01` (Ctrl-A 进入 raw REPL) → 等待 `raw REPL; CTRL-B to exit` 横幅，超时/不匹配抛 `RawReplError`；

- **执行协议**：写入 UTF-8 代码 + `\x04` (Ctrl-D) → 等待 `OK` 确认 → 读 `stdout` 至第一个 EOT (`\x04`) → 再读至第二个 EOT 得 `stderr` → `success = (len(stderr) == 0)`;
- **软复位**：执行 `import machine; machine.reset()`;
- **硬复位**：DTR/RTS 自动复位电路时序控制 (CP2102N 的 DTR/RTS 组合触发芯片复位)。
- **设计约束**：所有同步助手假设调用方先 `pause_monitor()` 独占串口，避免与后台监听冲突。

执行协议核心（节选自 `raw_repl.py` 的 `exec_raw`，省略超时恢复）：

```
ser.timeout = timeout
ser.write(code.encode("utf-8"))
ser.write(b"\x04")          # Ctrl-D: 执行
ok = ser.read_until(b"OK")  # 等待 OK 确认（随后紧跟 stdout）
if b"OK" not in ok:
    raise RawReplError("Raw REPL did not acknowledge code execution")
# 读 stdout 至第一个 EOT，读 stderr 至第二个 EOT，并剥离尾部 EOT
stdout_bytes = ser.read_until(b"\x04")
if stdout_bytes.endswith(b"\x04"):
    stdout_bytes = stdout_bytes[:-1]
stderr_bytes = ser.read_until(b"\x04")
if stderr_bytes.endswith(b"\x04"):
    stderr_bytes = stderr_bytes[:-1]
return {
    "success": len(stderr) == 0,
    "stdout": stdout_bytes.decode("utf-8", errors="replace"),
    "stderr": stderr_bytes.decode("utf-8", errors="replace"),
}
```

`exec_raw` 返回统一的 `{success, stdout, stderr}` 结构，供上层执行器、日志注入与错误解析直接消费。

选型理由：raw REPL 相比 `ampy` 等工具更底层、控制精确，且不引入额外依赖，直接用 `pyserial` 发送控制字节即可。

4.3.2 文件传输（`file_transfer.py`，119 行）

上传（`esp32_upload`）采用 base64 分块策略：本地文件整体 `base64.b64encode` 后按 256 字符/块切分，逐块在设备端执行 `open(path, 'ab') + ubinascii.a2b_base64(chunk)` 追加写入。选型理由：base64 保证二进制安全，256 字符块避免单条 REPL 命令过长导致传输失败。**下载**（`esp32_download`）则一条命令读取全文件并 base64 打印，主机端拼合解码写本地。相比 `toolchain_proposal.md` 初稿的「单条 `bytes(list(content))` 写入」方案，最终实现为「base64 分块」，是面向大文件可靠性的演进。

4.3.3 程序执行与复位 (`executor.py`, 100 行)

- `esp32_execute`: `run` 模式在 `raw` REPL 中执行 `exec(open(entry_file).read())`, 捕获 `stdout/stderr` 并注入日志缓冲 (`append_log()`), 使后续 `esp32_logs` / `esp32_error` 能立即看到输出; `stop` 模式发送 Ctrl-C 中断;
- `esp32_reset`: `soft` 走 `raw_repl.soft_reset` 并把 `boot` 输出追加进日志; `hard` 走 DTR/RTS 时序; 复位后等待 0.5 s 再恢复串口监听, 以捕获 MicroPython 启动横幅。

4.3.4 串口监控与日志检索 (`serial_monitor.py`, 158 行)

采用单例串口 + 环形缓冲区 + 后台 `asyncio` 任务架构:

- 后台任务用 `asyncio.to_thread` 包装阻塞式 `readline()`, 持续读取串口并写入 `deque(maxlen=1000)` 环形缓冲, 串口错误也记入缓冲;
- `esp32_serial` 支持 `start/stop/read` 三种动作, `read` 自动确保监听在运行, 睡眠指定时长后返回整个缓冲区快照;
- `esp32_logs` 取最近 N 行并按关键字过滤;
- 串口互斥设计: `pause_monitor()/resume_monitor()` 让其他工具操作串口前暂停后台监听、结束后恢复, 解决多工具并发访问串口冲突——这是工具链稳定性的关键设计。

4.3.5 错误解析 (`error_handler.py`, 82 行)

用正则逐帧匹配 MicroPython Traceback 结构 (`File "name", line N, in func`), 按 `Traceback (most recent call last):` 切块, 提取异常类型、消息、调用栈帧 (`file/line/function/code`), 输出结构化字典; `esp32_error` 从 `esp32_logs(lines=200)` 取日志, 默认只返回最后一个 Traceback, 把原始文本转化为结构化错误, 供 LLM 直接理解修复。

4.4 与 AI 编程助手的集成

KimiCode 集成方式 (README:209-218): 在 VS Code KimiCode 设置的 `mcpServers` 中配置三字段——`command` 为项目 `venv` 的 Python 解释器路径、`args` 为 `mcp_server.py` 路径、`env` 注入 `PYTHONPATH`。加载后 KimiCode 实测显示 "6 tools connected", AI 即可像调用本地函数一样调用硬件工具。

KimiCode 选型理由 (`docs/mini_claude_code_notes.md`): 国产模型中文理解与生成能力强、API 成本低、MCP 标准化、LLM 与工具解耦可单独开发测试。`mini_claude_code_notes.md` 同时提供了对 `mini-claude-code` 源码的深入理解 (`v0`→`v4` 版本演进、工具循环机制、Skill 机制), 是本工具链设计哲学的直接来源。

4.5 技术选型分析

候选方案	说明	评估结论
MCP 服务器（采用）	标准协议、生态成熟、工具可独立测试	推荐
Codex 插件	OpenAI 生态、绑定其客户端	生态封闭
Zcode 扩展	开源环境	资料较少
OpenCode + DeepSeek API	开源 + 低成本	工具扩展机制不成熟
自定义 CLI	简单但无法被 AI 标准调用	需自行封装

表 4-1 工具链技术选型对比

其余关键选型：`asyncio` + `后台任务`（串口监听不阻塞主线程、环形缓冲防内存溢出）、`正则解析 Traceback`（无需引入第三方反序列化库）、`base64 分块上传`（二进制安全与大文件可靠）。各候选方案的评估结论见表 4-1。

5 闭环验证与分析

5.1 闭环流程定义

任务书要求的 AI 原生闭环为「AI 发现问题 → 修改代码 → 自动部署 → 验证通过」。docs/toolchain_proposal.md 将其细化为 6 步工程管线：`写代码` → `upload` → `execute` → `serial 读输出` → `error 解析 Traceback` → `LLM 修复` → `循环直至验证通过`。本项目的工具链闭环验证以 tests/test_toolchain_w3.py 为自动化证据。

5.2 闭环验证实验

tests/test_toolchain_w3.py（140 行）在真实硬件上完成四步闭环（表 5-1），全部通过：

表 5-1 工具链闭环验证步骤（test_toolchain_w3.py）

步骤	操作	验证断言
① 上传	esp32_upload 上传含 <code>print("MCPiano W3 upload OK")</code> 的脚本	上传成功
② 执行 + 观测	esp32_execute 执行，esp32_logs 检索日志	日志中出现 "MCPiano W3 upload OK"
③ 错误诊断	上传 <code>raise ValueError(...)</code> 脚本并执行，esp32_error 解析	解析出 <code>type == "ValueError"</code> 且消息匹配
④ 复位	esp32_reset 软复位	日志中出现 boot 标记 <code>"rst:" / "boot:" / "entry 0x4"</code>

此外, `tests/test_server.py` 作为加载前冒烟测试: 以子进程启动 MCP 服务器, 向 `stdin` 发送 `initialize` (协议版本 2024-11-05) 与 `tools/list` 两个 JSON-RPC 请求, 验证 `stdio` 握手与工具可发现性。

三层证据链: ① 单元级 `tests/test_server.py` 验证 MCP 协议握手; ② 硬件级 `test_toolchain_w3.py` 验证上传→执行→诊断→复位闭环; ③ 集成级 KimiCode 实测 "6 tools connected" 并在 W4 视频中演示 AI 自主完成一次完整迭代 (README:323-325)。

5.3 闭环验证中遇到的问题与解决

1. **串口互斥冲突**: 多个工具并发访问串口导致数据竞争。解决: 后台监听与前台操作通过 `pause_monitor/resume_monitor` 互斥, 成为稳定性的关键设计。
2. **静默超时误判**: 设备无响应时初版会误判为程序死循环。后续定位为串口芯片驱动 (Yuanshen 环境所用 CH9102/CH340, 本仓库为 CP2102N) 不强制独占串口导致的死信道问题, 通过串口故障自诊断区分原因 (在 Yuanshen 的 `esp32_piano_mcp.py` 中完善)。
3. **大文件传输失败**: 初版单条命令写入方案对大文件不可靠, 改为 base64 256 字符分块后稳定。
4. **硬件级问题 (钢琴侧)**: 八度键 INPUT-ONLY 浮空误触发、MAX98357A GAIN/SD_MODE 接错导致削顶与失声——均通过文档化接线规范与外部上拉解决。

5.4 工具链对开发效率的影响分析

人工迭代一次「上传—执行—观测—诊断」约需 1-2 分钟且需全程盯串口; 工具链将单次闭环压缩到秒级 (raw REPL 单次执行约 0.1 s, Yuanshen 实测), 且错误解析把原始 Traceback 结构化, LLM 可无歧义理解。整个 W4 的 AI 闭环演示中, AI 在无人干预下自主完成了发现 bug → 修复 → 重部署 → 验证通过的全过程, 验证了任务书所称“人充当 AI 与硬件之间传声筒”的模式确实可以被打破。

6 AI 协作反思

6.1 本项目中的 AI 应用方式

本项目的 AI 应用经历了三个层次:

1. **代码生成助手 (W1-W2)**: 用 AI 编写测试程序与钢琴固件, 人工验证。典型如 W1 外设测试、W2 九键钢琴 v1, AI 生成代码后由工程师在硬件上验证修正。
2. **工具链驱动 (W3-W4)**: AI 通过本项目 MCP 工具链自主完成部署—观测—诊断—修复闭环 (详见第 5 章)。
3. **智能体自动化 (W4)**: 小组自主设计闭环智能体 Yuanshen, 把整个开发流程 (需求规范化→接线确认→编码→烧录→实机测试→汇报) 交给 Agent 执行。

6.2 小组智能体协作成果：Yuanshen

Yuanshen（仓库 `github.com/WANG5294/Yuanshen`, commit `cbd9bc7`, v1.2.3）是基于 mini-claude-code 思想自主设计的面向 ESP32 MicroPython 开发的闭环自动化 Agent，Python 代码约 4479 行，npm 包 `yuanshen-esp32-agent` 已发布（10 个版本，latest 1.2.3，维护者含本组成员）。本节数据均据远程仓库 commit `cbd9bc7` 实际代码统计，结构见表 6-1。

整体架构（表 6-1）：主 REPL 入口 + 11 模块核心包 + 设备 MCP 服务器 + A2A 服务。

表 6-1 Yuanshen 仓库结构（41 个文件，7850 行）

文件/目录	行数	职责
<code>yuanshen.py</code>	551	主入口：斜杠命令分发、主 REPL 循环、自由对话
<code>esp32_piano_mcp.py</code>	802	设备 MCP 服务器，17 个工具（12 设备通道 + 5 音频闭环）
<code>a2a_server.py</code>	239	A2A 服务端（JSON-RPC over HTTP, Agent Card）
<code>a2a_client_mcp.py</code>	148	A2A→MCP 桥，供 Kimi Code 委派任务
<code>yuanshen/</code> 包	2229	agent 闭环核心 / UI / LLM 客户端 / 工具路由 / Todo 状态机 / 提示词 / MCP 客户端 / Skill
<code>skills/</code>	1050	11 个 ESP32 硬件 Skill（SKILL.md）
<code>docs/</code>	1562	架构、Prompt 缓存、安全审计、862 行硬件手册等

闭环流程的代码实现（对应任务书要求的完整闭环）：

1. **需求规范化**：独立 LLM 调用（`NORMALIZE_SYSTEM` 8 条规则 + `submit_normalized_task` 工具），注入用户输入 + `wiring.md` + 862 行硬件手册，强制提交 `requirement/wiring/audio_required` 三字段，失败自动重试 3 次；
2. **接线确认门禁**：左右分栏 + `difflib` 变更高亮展示规范化草案，人工确认（1 确认/2 修改/3 切串口/4 取消）前不写入 `wiring.md`、不进入 Agent 循环；
3. **Agent 主循环**：流式调用 LLM（token 级渲染）→ 重建统一响应（`thinking` 块含 `signature` 原样回传）→ 执行工具（6 本地 + 17 MCP）→ 追加历史 → 最新 `ToDoList` 置末 → 轮次卡片展示 + `rounds/` 审计快照落盘，`MAX_ITERATIONS = 100`；
4. **烧录把关**：`upload` 先写 `.dest.uploading` 临时文件、512 字节分块、设备端 SHA-256 校验通过才备份替换为 `main.py`，全程可回滚；Todo 状态机硬性要求未部署 `main.py` 不得标记"烧录"完成；

5. **实机测试验收**：音频闭环 `mic_check` → `play_and_record` → `analyze_wav` → `compare_audio`，按量化指标验收（峰值/噪声底比 ≥ 5 、音高偏差 ≤ 10 音分、包络相关 ≥ 0.8 ）；`analyze_wav` 用纯 numpy FFT 主峰 + 抛物线插值测基频；
6. **结果汇报**：最终报告渲染 + 归档 + `extract_skill()` 把流程经验提炼为 `exp-* Skill`（用户确认后保存、热重载即时可用）。

关键技术点：双 provider 模型注册表（DeepSeek 走 Anthropic 兼容端点、Kimi 走 OpenAI 端点，`/model` 命令切换）；自研 152 行 MCP stdio 客户端；受限 shell（禁止管道/重定向/复合命令 + 命令白名单）；`safe_mcp_path()` 路径围栏；A2A 任务下发（远程任务仍保留本机人工确认、单任务锁、busy 即 failed、final_report artifact 回传）；Rich 终端 UI（ASCII logo、四步进度条、轮次卡片、prompt_toolkit 补全）。

6.3 AI 的贡献与局限（具体案例）

贡献案例：

- **I2S 音频升级**（W2）：AI 生成 MAX98357A 驱动与正弦波 PCM 编码，人工修正 GAIN/SD_MODE 接线后一次通过；
- **真实采样回放**（W4）：AI 实现 512B 分块流式播放与 WAV 头解析，配合人工录制的 21 个钢琴采样完成音色升级；
- **闭环自动化**（W4）：KimiCode 通过工具链自主完成发现 bug→修复→重部署→验证的完整迭代（W4 视频演示）；
- **Yuanshen 架构**：大量代码由 AI 辅助生成，人工负责方向决策与验证（W4 汇报实录："人工确定方向，AI 生成代码，再进行人工验证"）。

局限与不足（如实记录）：

1. **API 漂移**：`test_piano_v1.py` 引用 W2 旧版 Piano API，与 W4 采样版固件不兼容（ImportError），README 快速开始文档未同步——AI 与人工都未及时回归测试；
2. **工具清单不一致**：`esp32_download` 已实现未注册，README 列 7 项实际 6 项，说明"实现"与"发布"之间缺少自动化校验；
3. **AI 无法自获硬件信息**：任务书强调工程师必须提供准确硬件上下文，本项目通过 wiring.md + 硬件手册注入缓解，但"AI 自主感知硬件状态"仍是未完全解决的问题；
4. **A2A 局限**：W4 汇报实录"A2A 功能局限于实现演示功能"，远程任务仍依赖本机人工确认、无认证、不支持流式与取消，实际价值有待第五周进一步探索；
5. **阻塞式播放**：多键连接按顺序覆盖播放，无法同时发声（硬件层面亦如此），影响表现力。

6.4 对 AI 辅助嵌入式开发的思考

- **AI 在代码生成 vs 工具链开发中的表现不同**：代码生成场景 AI 输出质量高但需人工兜底验证；工具链（协议、状态机、并发）场景 AI 能快速给出可行方案，但协议细节（raw REPL 时序、DTR/RTS 时序）仍需工程师依据 datasheet 校准。
 - **验证是唯一标准**：AI 生成的每一行代码都必须在物理硬件上验证（本项目坚持"硬件验证是唯一标准"）。
 - **工具链让 AI 从"建议者"变为"执行者"**：一旦 AI 掌握与硬件的通信能力，迭代速度从"分钟级人工循环"提升到"秒级自动循环"，这是 AI 原生嵌入式开发的本质价值。
-

7 总结与展望

7.1 主要成果

1. **任务一（数字钢琴）**：基于 ESP32 + MicroPython 的 9 键数字钢琴，具备 21 个真实钢琴采样（A3-G5）流式回放、三档八度切换（LED 指示）、录制/回放、LED 硬件直驱零延迟声光同步，5 周渐进式开发，34+ 条提交记录。
2. **任务二（工具链）**：基于 MCP 协议的 6 工具链（上传/执行/复位/串口监控/日志/错误解析），raw REPL 协议层 + 串口互斥 + Traceback 结构化解析，KimiCode 集成实测通过，并在真实硬件上完成四步闭环验证。
3. **AI 原生闭环验证**：AI 通过工具链自主完成「发现问题→修改代码→自动部署→验证通过」完整迭代。
4. **智能体延伸（小组协作）**：Yuanshen 闭环自动化 Agent（4479 行 Python、17 个 MCP 工具、A2A 桥接、Skill 沉淀、npm 发布 v1.2.3）。
5. **文档体系**：README、hardware/bom_analysis.md（38 项 BOM）、docs/ 三份设计文档、周报视频 4 次。

7.2 经验教训

- 硬件文档是权威参考，GPIO/引脚特性（INPUT-ONLY、SPI Flash 占用）必须从原理图核实，不可臆测；
- 串口互斥与故障自诊断是工具链稳定性的生命线；
- 文档必须与代码同步演进（API 漂移教训）；
- AI 协作中"人工确定方向、AI 生成代码、人工验证"是当前最有效的协作模式。

7.3 未来改进方向

- 注册 esp32_download 补齐 7 工具，增加工具清单自动校验（测试断言 tools/list 与文档一致）；
- 多段录音/循环播放/动画等扩展功能（README W5 计划）；

- Yuanshen: A2A 增加认证与流式支持、探索多 Agent 协作、把硬件状态感知（自动扫描 GPIO 拓扑）纳入闭环；
- 数字钢琴扩展：支持和弦（多通道 I2S 混音）、力度感应、旋律自动播放模式。

附录 A 周度开发历程（git 提交记录摘要）

周次	时间段	主要提交（commit 缩略）	里程碑
W1	7/10–7/11	180b9d7 870e4f0	硬件分析文档、外设测试、mini- claude-code 阅读笔记
W2	7/14–7/17	af812c0 → 6f3d284 → e0994aa	PWM→I2S 功放升级、9 键钢琴 v1、工 具链 6 工具 + KimiCode 集成；OLED 故障移除（48d07f4）
W3	7/20–7/26	0e5a879	GPIO5 录制/播放扩展（README 归入 W3；提交日期为 7/28）；LED 硬件直驱 为本周成果
W4	7/27–8/2	ee2e148 4a68745	21 个真实钢琴采样回放、录制/回放移 植、八度 toggle + LED 指示、 Yuanshen 项目
W5	8/3–8/9	（本报告）	最终演示视频、技术报告、仓库整理

附录 B 测试清单

测试脚本	验证对象	阶段
tests/test_button.py	GPIO34/35 电平/边沿/去抖计数	W1
tests/test_led.py	GPIO32/33 LED	W1
tests/test_buzzer.py	PWM 蜂鸣器（后已被 I2S 功放取代）	W1
tests/test_buttons_9key.py	9 键矩阵扫描	W2
tests/test_max98357a.py	I2S 功放正弦波/音阶/音量	W2
tests/test_piano_v1.py	钢琴 v1 集成（与 W4 固件 API 不同步）	W2
tests/test_octave_with_led.py	八度键与 LED 联动	W3
tests/test_toolchain_serial.py	串口监控/日志工具（真机）	W3
tests/test_toolchain_w3.py	工具链闭环：上传→执行→诊断→复位	W3
tests/test_server.py	MCP stdio 握手 + tools/list 冒烟	W3

附录 C 参考资料

1. 25 级实验班暑假大作业任务书 V2: 《AI 原生嵌入式系统编程综合性实验项目任务书》 (`docs/最新25级实验班暑假大作业要求.pdf`)
2. `hardware/bom_analysis.md`: 硬件资源分析 (原理图/PCB/BOM)
3. `docs/toolchain_proposal.md`: MCP 服务器实现指南
4. `docs/toolchain_architecture.md`: 工具链架构设计
5. `docs/mini_claude_code_notes.md`: mini-claude-code 源码分析
6. MicroPython 官方文档: raw REPL 协议
7. MCP (Model Context Protocol) 规范: `modelcontextprotocol.io`
8. Yuanshen 仓库文档: `docs/architecture-v4.md`、`docs/code-audit-v1.0.md`、`docs/reference/修正ESP32_D0WD_硬件开发手册.md`